



Ember Vaults Audit

audited by Asymptotic

December 10, 2025

Summary

Asymptotic audited the Ember Vaults protocol and identified 9 issues: 2 medium-severity findings related to fee calculation and timing manipulation risks, and 7 low-severity issues covering slippage protection, input validation, and deployment concerns. All issues were addressed to our satisfaction, and we are confident in the protocol's security posture.

Ember Vaults operates as a managed fund. Operators can withdraw any amount from the vault to deploy to whitelisted sub-accounts. Rate managers update exchange rates to reflect off-chain portfolio values. The protocol enforces share accounting, FIFO withdrawal ordering, and rate change limits on-chain. It cannot verify fund deployment or validate rate accuracy.

Three roles control the system: operators manage fund deployment and process withdrawals; rate managers set share exchange rates; admins assign these roles and can pause operations. The protocol does not enforce withdrawal processing times, track external strategy performance, or verify that rates match actual holdings. Fund allocation and strategy selection occur off-chain through operator decisions rather than programmatic rules.

Withdrawal requests are served at the current rate when processed, not when created. Users should consider the maximum possible rate change between request and fulfillment. Rate change limits are per pool and set at pool creation.

Audited code

We audited the following directories at the given commit:

- sources

Initial Commit: 269540870ac9d5e59a2c191df8c3ed6890ef9773

Final Commit: e103f490f5e1f73ef826852fef411fb03123486d

Final Report Date: December 10, 2025

Status: Final

Legend

Issue severity

- **Critical** — Vulnerabilities which allow account takeovers or stealing significant funds, along with being easy to exploit.
- **High** — Vulnerabilities which either can have significant impact but are hard to exploit, or are easy to exploit but have more limited impact.
- **Medium** — Moderate risks with notable but limited impact.
- **Low** — Minor issues with minimal security implications.
- **Advisory** — Informational findings for security/code improvements.

Legal Disclaimer

Terms and Conditions and Liability

This report is subject to the terms and conditions—including liability limitations—established between Asymptotic and the paying entity. By sharing code with Asymptotic, developers acknowledge and agree to these conditions. In the absence of executed agreements, liability is limited to the fees paid for these services.

Scope

This security audit report focuses specifically on reviewing the Move smart contract code. The audit does not analyze or make any claims about other components of the system, including but not limited to:

- Frontend applications and user interfaces
- Backend services and infrastructure
- Off-chain components and integrations
- Deployment procedures and operational security
- Third-party dependencies and external services

Limitations

The findings and recommendations in this report are limited to the Move code implementation and its immediate interactions within the Sui blockchain environment.

Creating proofs for specifications is on a "best effort" basis. We manually audited the code in instances where formal proofs were not technically feasible.

M-1: Incorrect Fee Denominator Causes Fee Overcharge

Severity: **Medium**

Description

The `charge_accrued_platform_fees` function calculates platform fees using an incorrect denominator constant. The `FEE_DENOMINATOR` is set to `31_536_000_000_000_000`, but based on the comment and formula requirements, it should be 1000x bigger.

The fee formula $\text{fee} = \text{tv1} \times \text{fee_percentage} \times \text{elapsed_time_ms} / \text{FEE_DENOMINATOR}$ divides by this denominator and produces result 1000x higher than the configured fee percentage suggests.

For the yearly fee percentage, the constant should be: `number of milliseconds in a year * the fee denominator` (100That is exactly what comment contains: `(365 * 24 * 60 * 60 * 1000) * 1e9` This equals `31_536_000_000_000_000_000`).

The test file copies the same incorrect constant and uses it in `calculate_expected_fee()` helper. Tests compare actual output against expected values computed with the same flawed formula, creating circular validation that masks the bug.

Notice: the operator is responsible for fee collection and for maintaining sufficient balances to enable fee extraction. Since the operator also manages liquidity transfers for yield generation, this role already has elevated trust and privileges. Therefore the issue severity is downgraded, since only a malicious or compromised operator could cause direct financial harm.

Recommendation

Update `FEE_DENOMINATOR` to the mathematically correct value.

Add an economic test that validates the formula and constants by asserting the hardcoded scenario. For example: "1000 USDC TVL with a 1% annual fee over 365 days should accrue exactly 10 USDC in fees".

Remediation

✓ Remediated

✓ Commit: e103f49

M-2: Manual Fee Charging Creates Timing Manipulation Risk

Severity: **Medium**

Description

The platform fee mechanism uses snapshot-based calculation instead of time-weighted accrual. The `charge_platform_fee` function calculates fees by simply multiplying the current TVL by the fee percentage, without considering how long assets have been in the vault or how TVL changed over time.

The fee amount depends entirely on TVL at the exact moment the operator calls `charge_platform_fee`, not on the actual TVL throughout the entire period.

The operator can choose when to charge fees to their advantage. For example, charging before withdrawals captures higher TVL, charging after captures lower TVL.

The calculation doesn't factor in time. Charging fees after 1 day looks the same as charging after 365 days if TVL hasn't changed, which makes no sense economically.

The operator needs to call `charge_platform_fee` function daily. If they forget or encounter technical issues, the fees for that period are permanently lost and not accumulated for future collection. While it's technically possible to work around this by increasing the fee rate before the next charge, this approach is unreliable since the TVL may fluctuate and makes the logic less robust.

Overall, protocol likely collects less fees than intended.

Recommendation

Switch from snapshot-based to time-weighted fee accrual.

Track a `last_charge_timestamp` to record when fees were last calculated, and update it each time the fee is charged. To determine the fee amount, apply the annual fee rate proportionally to the time elapsed since the last charge.

Trigger fee charging automatically before any state-changing operations (such as deposits, withdrawals, or rate updates), rather than relying on a manual operator call. And make the fee-charging function internal (non-public), as it will now be invoked programmatically within the protocol logic.

Remediation

✓ Remediated

✓ Commit: e103f49

L-1: Fee Settlement Occurs After State Changes, Missing Fees on Historical TVL

Severity: **Low**

Description

Three functions charge fees after modifying state that affects the fee calculation, causing fees to be computed on the new state rather than the state that existed during the fee period:

- Rate updates (`update_vault_rate`) directly affect the TVL used for fee calculation. Historical period gets charged at the new rate's TVL.
- During withdrawal processing (`process_withdrawal_requests`, `process_withdrawal_requests_u`) fees are charged after shares are burned, reducing TVL before fee calculation. Shares that existed during the fee period are excluded from the fee calculation.

Recommendation

Move `charge_accrued_platform_fees` to execute before any state modifications in all three functions.

Remediation

✓ Remediated

✓ Commit: e103f49

L-2: Missing Slippage Protection During Depositing

Severity: **Low**

Description

The `deposit_asset()` and `mint_shares()` functions lack slippage protection, exposing users to rate changes before transaction execution.

In `deposit_asset`, users deposit a fixed token amount but receive a variable number of shares calculated as $\text{shares} = \text{amount} \times \text{rate}$. If the rate decreases before execution, users receive fewer shares than expected.

The vault rate can be updated by the `rate_manager` via `update_vault_rate`. While rate changes are constrained by time intervals and maximum percentage limits, they can still occur between when users see the rate on the frontend and when their transaction executes on-chain.

In contrast, `mint_shares` has implicit slippage protection. The user supplies a fixed `Coin<T>` amount and requests a fixed number of shares. If the rate decreases and the required token amount exceeds what the user provided, the transaction reverts with `EInsufficientBalance`.

Recommendation

Add slippage protection to the `deposit_asset` function by introducing a `min_shares_out` parameter.

Remediation

✓ Remediated

✓ Commit: e103f49

L-3: Missing Zero Address Validation in Sub-Account Management

Severity: **Low**

Description

The `set_sub_account` function allows adding the zero address `@0` to the `sub_accounts` whitelist without validation. This enables accidental or erroneous fund loss when the operator later withdraws funds to the zero address.

Recommendation

Add zero address validation in the `set_sub_account` function.

Remediation

✓ Remediated

✓ Commit: a939496

L-4: Version Upgrade Requires Sequential Deployment

Severity: **Low**

Description

The `increase_supported_package_version` function increments the stored version by 1 rather than setting it to the package `VERSION` constant. This forces sequential version upgrades and can cause temporary operational issues if version numbers are skipped during deployment.

Current logic:

- Checks: `config.version < VERSION`
- Updates: `config.version = config.version + 1`

Example scenario:

- protocol at version 1
- deploy package version 3
- the check passes ($1 < 3$),
- but `config.version` becomes 2 instead of 3.

Since the package `VERSION` is 3, `verify_supported_package` fails until admin increases the version again to sync the version. If users aren't supposed to call version 2, the upgrade from v1 to v3 must be done in a PTB with sequential version increments, preventing any interaction with version 2.

Recommendation

Update version synchronization to match deployed package version directly instead of incrementing.

Alternatively, if strict sequential upgrades are desired, add explicit check before assignment to ensure old version differs only by 1.

Remediation

Acknowledged

L-5: Incomplete Blacklist Validation in Withdrawal Request Creation

Severity: **Low**

Description

The `redeem_shares` function only validates that the owner (sender) is not blacklisted, but does not check the receiver address. However, `process_request` validates both owner and receiver for blacklist status.

This inconsistency allows users to create withdrawal requests with blacklisted receivers, causing shares to be locked unnecessarily until the request is processed and subsequently skipped. This creates a poor user experience where users can queue withdrawal requests that will inevitably fail.

Recommendation

Add receiver blacklist validation in `redeem_shares` function immediately after the owner check.

Remediation

✓ Remediated

✓ Commit: a939496

L-6: Missing Role Separation Validation for Privileged Addresses

Severity: **Low**

Description

The vault does not enforce separation of duties between privileged roles, allowing a single address to simultaneously hold multiple or all of the following roles: admin, operator, rate_manager, and sub_accounts.

Current validation in role management functions only checks that addresses are non-zero and different from their current values to prevent no-op updates. No function validates that these roles are held by different addresses.

Recommendation

Add role separation validation to prevent a single address from holding multiple privileged roles.

Remediation

✓ Remediated

✓ Commit: e103f49

L-7: Fee Percentage Change Without Prior Fee Settlement

Severity: **Low**

Description

The `update_vault_fee_percentage` function changes the fee rate immediately without first settling accrued fees at the old rate. Since `charge_accrued_platform_fees` calculates fees using the current `vault_fee_percentage` multiplied by the entire elapsed time since last charge, any rate change retroactively applies to the historical period.

Recommendation

Add clock parameter to `update_vault_fee_percentage` and settle fees before updating value in vault.

Remediation

✓ Remediated

✓ Commit: e103f49

A-1: Missing Upper Bound on Maximum Rate Interval

Severity: **Advisory**

Description

The `update_max_rate_interval` function validates that the new value is greater than or equal to `min_rate_interval` but does not enforce an upper bound. This allows setting arbitrarily large intervals that could temporarily restrict rate update operations.

If admin sets `max_rate_interval` to an extremely large value like `u64::max_value!()`, any vault created with that interval cannot have its rate updated until the vault admin reduces the interval via `change_vault_rate_update_interval`.

Recommendation

Add a reasonable upper bound check for `max_rate_interval`.

Remediation

✓ Remediated

✓ Commit: a939496

A-2: Missing No-Op Check in Pause Function

Severity: [Advisory](#)

Description

The `pause_non_admin_operations` function does not verify whether the new pause state differs from the current state, allowing no-op operations that waste gas.

All other configuration update functions prevent no-ops.

Recommendation

Add a no-op check consistent with other update functions.

Remediation

✓ Remediated

✓ Commit: `a939496`

A-3: Strict Inequality in Time Comparisons Requires Extra Wait

Severity: [Advisory](#)

Description

Three time-based validation functions use strict inequality $>$ instead of inclusive comparison $\geq$, requiring users to wait at least one additional millisecond beyond the stated interval:

- `update_vault_rate` checks `current_time - last_updated_at > rate_update_interval`. Must wait `interval + 1ms`.
- `charge_platform_fee` checks `current_time - last_charged_at > ONE_DAY_MS`. Must wait `1 day + 1ms`.

The 1 ms offset is minor but causes inconsistency between expected and actual behavior, and makes the function aborted when called exactly at the boundary time.

Recommendation

Change strict inequality $>$ to inclusive comparison $\geq$ in all three locations for more intuitive behavior.

Remediation

✓ Remediated

✓ Commit: a939496

A-4: Asymmetric Percentage Calculation Causes Unequal Rate Constraints

Severity: [Advisory](#)

Description

The `diff_percent` function is asymmetric due to dividing by the first parameter only:

`diff_percent(a, b) = |a - b| / a`

This mathematical property creates directionally dependent results:

- `diff_percent(100, 50) = 50 / 100 = 50.00%`
- `diff_percent(50, 100) = 50 / 50 = 100.00%`

It measures “percentage change FROM the first value”.

The function used in vault rate updates only, validating that the new rate doesn’t deviate more than the allowed percentage from the old rate. The constraint is unidirectional.

In practice, rate decrease for 50% will require increase 100% to set rate back. The asymmetry means rate decreases create “traps” where the rate manager cannot fully reverse the change within the same percentage limit, requiring multiple smaller updates to recover.

Recommendation

Change function name to to clarify directional nature, e.g. `percent_change_from(old_value: u64, new_value: u64)`

Consider symmetric alternative to ensure if rate can change $A \rightarrow B$ with limit $X\%$, it can also change $B \rightarrow A$ with the same limit $X\%$.

Remediation

✓ **Remediated**

✓ **Commit:** a939496

Notes: The function has been renamed to `percent_change_from`.

A-5: Inefficient Ceiling Division Implementation

Severity: **Advisory**

Description

The `div_ceil` function computes the same intermediate multiplication twice, leading to unnecessary computational overhead. It calculates $(a \times \text{BASE}) \div b$ for the result, and then recalculates $(a \times \text{BASE}) \bmod b$ for the remainder. The intermediate value $(a \times \text{BASE})$ is computed redundantly.

Recommendation

Apply the mathematical trick $(a + b - 1) \div b$ which eliminates both the modulo operation and the conditional branch, resulting in cleaner, faster code. This formula is equivalent to $\text{ceil}(a/b) = \text{floor}((a + b - 1) \div b)$.

Remediation

✓ Remediated

✓ Commit: a939496

A-6: Incorrect and Misleading Documentation

Severity: **Advisory**

Description

Multiple functions have documentation that contradicts their actual implementation, creating confusion and potential for integration errors:

- pause check documentation: functions `change_vault_admin`, `change_vault_operator`, and `update_vault_fee_percentage` document pause checks that don't exist in the code
- operator-facing operations check only the protocol pause flag, but the documentation appears to be copy-pasted and incorrectly mentions "or the vault is paused"
- `set_blacklisted_account` comment says "true for whitelisting, false for blacklisting" but actual behavior is completely backwards
- Many functions don't document all possible error codes

Recommendation

Ensure that the documentation accurately reflects the code logic, or update the implementation if it does not align with the intended behavior.

Remediation

✓ Remediated

✓ Commit: a939496

Notes: Documentation is improved, but some inconsistencies still remain, e.g., the comment for `set_blacklisted_account` function is still unchanged.

A-7: Hardcoded Magic Numbers in Admin Validation and Defaults

Severity: [Advisory](#)

Description

Validation and defaulting logic in admin module use raw numeric literals instead of named constants, reducing clarity and causing a functional limit:

- defaults hardcoded in init (rates, intervals, fee)
- `update_max_fee_percentage` hardcodes 100% as 10000000000
- `update_min_rate_interval` hardcodes 1 minute as `60 * 1000`.

The `math::BASE` constant (representing 100%) exists but is only used within the math module and not reused here. If the admin module defines a different base value, all related calculations will become incorrect.

Recommendation

Replace hardcoded values with defined constants, and most importantly, consolidate all fee base logic to use a single shared constant.

Remediation

✓ Remediated

✓ Commit: a939496

Notes: Only init function is fixed.

Both 100% constants are still defined independently. Changing one without updating the other will lead to incorrect calculations.

A-8: Duplicate Error Code 0 Across Utility Modules

Severity: [Advisory](#)

Description

Both `math.move` and `queue.move` define error code 0.

- admin module codes start from 1000
- vault module codes start from 2000

While Move error codes are module-scoped (no runtime collision), the duplicate 0 reduces observability in logs and monitoring.

Recommendation

Adjust `queue.move` error codes to avoid overlap with `math.move`.

Remediation

✓ Remediated

✓ Commit: a939496

About the Auditor

Asymptotic provides a white-glove, formal-verification-based auditing service for Sui smart contracts.

Asymptotic Team

Lead Auditor

Mykhailo Burakhin — Principal Auditor

- Conducted detailed security audits and gas optimizations for complex DeFi applications
- Developed and audited ERC-20, ERC-721, and ERC-1155 token contracts, including staking, vesting, auctions, and launchpads
- Engineered cross-chain bridges using ERC-4337 account abstraction, multi-signature wallets, and upgradeable proxy contracts
- Implemented protections against anti-sniping and front-running, and secure onboarding with MPC and zero-knowledge proofs
- Over 15 years of experience architecting secure, high-performance systems with a strong focus on cryptography, blockchain security, and advanced C++ engineering